Laboratory Assignment 2: Parallel Programming with OpenMP
Solving the Heat Equation

Alessandro Fazio
alessandro.fazio@estudiantat.upc.edu
Jose Ricardo Arias Perez
jose.ricardo.arias@estudiantat.upc.edu

Group: M2-It1-gr11
Course 2026/27, Q1

Concurrency, Parallelism and Distributed Systems (CPDS)
FIB – Universitat Politècnica de Catalunya

October 26th, 2026

Contents

1 Sequential Jacobi and Gauss-Seidel codes **3**

2 Dependency analysis with Tareador **3**
2.1 Jacobi 3
2.2 Gauss-Seidel 8

3 Parallel Jacobi **13**
3.1 Data decomposition implementation 13
3.1.1 Overall analysis 14
3.1.2 Detailed analysis 15
3.2 Optimisation 16
3.2.1 Overall analysis of the optimised code 17
3.2.2 Detailed analysis of the optimised code 19

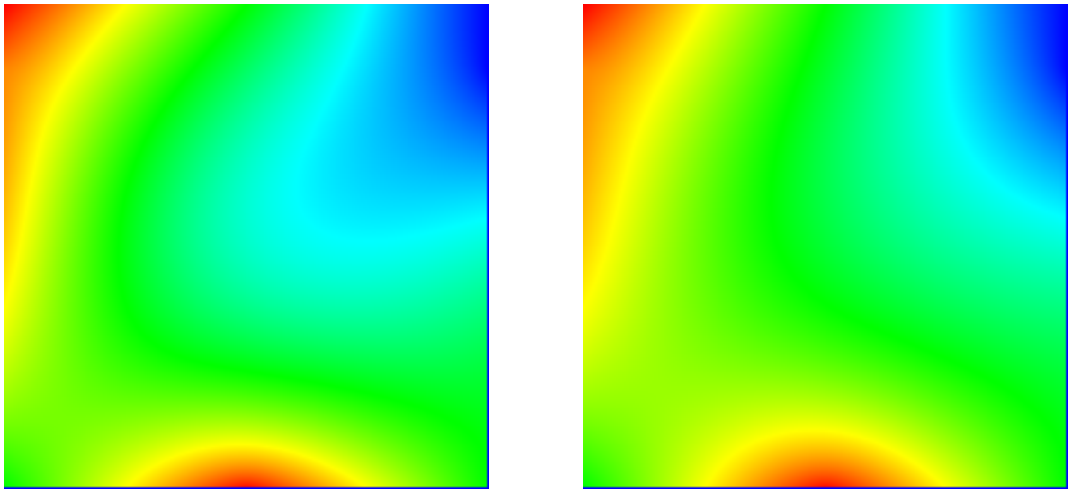
1 Sequential Jacobi and Gauss-Seidel codes

The program simulates heat diffusion in a 2D body with two heat sources, one in the upper-left corner and one in the middle of the lower border. Both solvers use the same function `solve`, which applies a 5-point stencil to every interior point:

- **Jacobi** reads `u` and writes the new values into a second matrix, `uhelp`. After each iteration, `copy_mat` copies `uhelp` back into `u`.
- **Gauss-Seidel** updates `u` in place, so each point uses values of its upper and left neighbours that were already updated in the same iteration.

The main loop stops when the residual drops below `param.residual` or when `param.maxiter` is reached. With `test.dat`, Jacobi converges after 15 756 iterations.

Figure 1 shows the images produced by the sequential code. They are the reference for validating the parallel versions: every parallel Jacobi image in this report is byte-for-byte identical to `heat-jacobi-seq.ppm` (checked with `cmp`). The two solvers produce slightly different images, so each parallel version is always compared with the image of its own solver.



(a) Jacobi (b) Gauss-Seidel

Figure 1: Images generated by the sequential code (`heat-seq`).

2 Dependency analysis with Tareador

Tareador runs the program once sequentially. For every region we mark as a task, it records which memory addresses the task reads and writes. If one task writes an address that another reads or writes (or reads an address that another writes), Tareador adds a dependency edge between them. From the resulting task dependency graph (TDG) it simulates the parallel execution on N processors, which gives an upper bound on the speedup of each task decomposition, without any runtime overheads.

The analysis uses the small problem from `run-tareador.sh` (`-s 32 -n 4`: a 32×32 body and 4 iterations). The accumulation into `sum` is excluded with `tareador_disable_object(&sum)`. Otherwise every task would depend on the previous one through `sum +=`; in the OpenMP code this conflict is resolved with a `reduction`.

2.1 Jacobi

We explored three task decompositions:

J1 (coarse): one task per call to `solve` (`jacobi`) and one per call to `copy_mat`. These are the default tasks in `heat-tareador.c`.

J2 (blocks in `solve`): J1 plus the `solve(%d,%d)` tasks in `solver-tareador.c`. Each call to `solve` is split into $4 \times 4 = 16$ block tasks; `copy_mat` remains a single task.

J3 (blocks in `solve` and `copy_mat`): J2 plus the `copy(%d,%d)` tasks, so the copy is also split into 16 block tasks.

Task dependency graphs. In J1 (Figure 2) the tasks form a single chain: `jacobi` → `copy_mat` → `jacobi` → ...

- `copy_mat` reads `uhelp`, which `solve` has just written.
- The next `solve` reads `u`, which `copy_mat` has just written.

The edges from each `jacobi` task back to the main program correspond to `residual`, which the `while` loop must read before starting the next iteration. The iterations of the main loop are therefore inherently sequential, and any parallelism has to come from inside each iteration.

In J2 and J3 (Figure 3) there are *no edges between the 16 `solve` blocks*. Each block reads `u` and writes only its own part of `uhelp`, so the blocks are independent. The same holds for the 16 `copy` blocks in J3. Between the phases there is a single edge (`jacobi` → `copy_mat` → `jacobi`), because the block tasks are nested inside the coarse tasks: every block of one phase must finish before any block of the next phase can start.

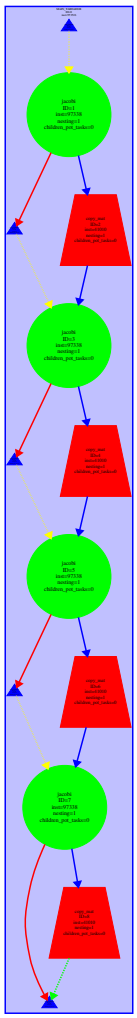
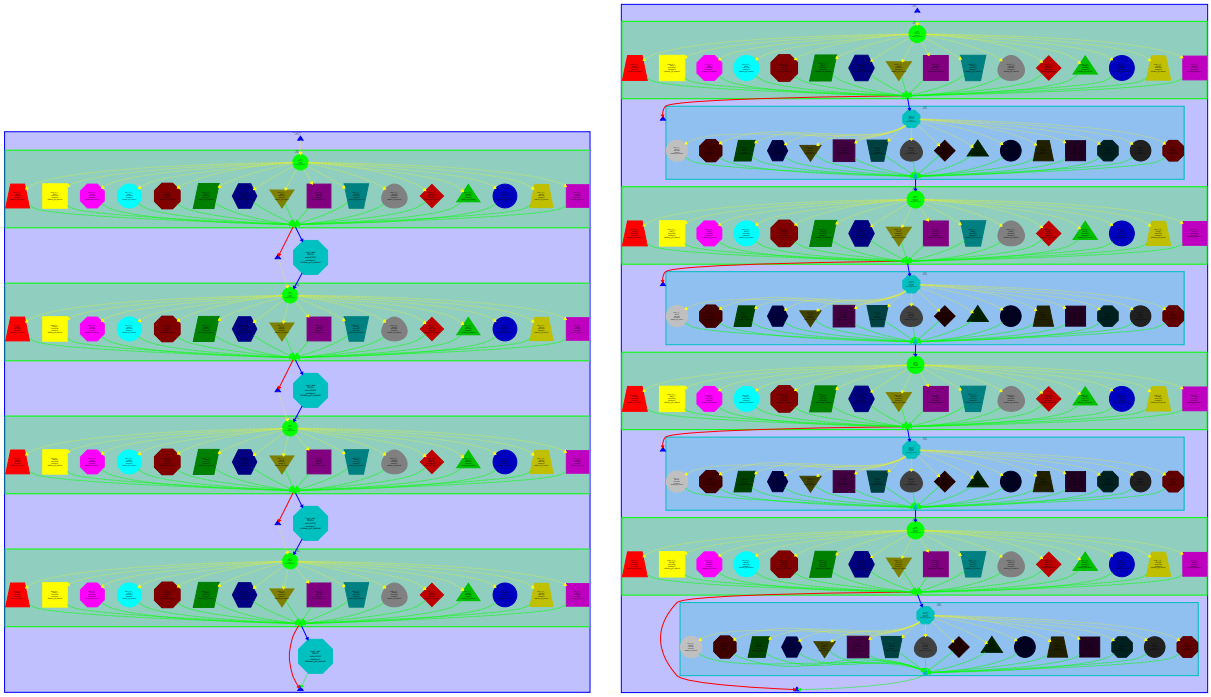


Figure 2: TDG of version J1 (coarse tasks), 4 iterations: a fully sequential chain.



(a) J2: 16 `solve` blocks per iteration, `copy_mat` as a single task.

(b) J3: 16 `solve` blocks and 16 `copy` blocks per iteration.

Figure 3: TDGs of the block decompositions. There are no dependencies between blocks of the same phase.

Simulated parallel execution. Table 1 shows the simulated execution time on 1, 4, 8 and 16 processors. Figures 4–6 show some of the timelines.

Version	1 proc.	4 proc.	8 proc.	16 proc.
J1 coarse	553.5 ms	553.4 ms	553.4 ms	553.4 ms
	1.00	1.00	1.00	1.00
J2 blocks (solve)	553.7 ms	264.1 ms	216.5 ms	196.1 ms
	1.00	2.10	2.56	2.82
J3 blocks (solve + copy)	553.9 ms	142.8 ms	75.4 ms	46.3 ms
	1.00	3.88	7.34	11.96

Table 1: Tareador simulation: simulated time (first row of each version) and speedup with respect to 1 processor (second row).

- **J1** has no parallelism at all. Figure 4 shows a staircase where only one processor is busy at any time, so the speedup is 1 for any number of processors.
- **J2** parallelises `solve`, but `copy_mat` runs on a single processor while all the others are idle: the long cyan bar in Figure 5. The copy is $4 \times 41\,010 = 164\,040$ of the 553 718 instructions, a serial fraction of $f \approx 0.30$. Amdahl's law limits the speedup to $1/f \approx 3.4$, and to $1/(0.30 + 0.70/16) \approx 2.9$ on 16 processors, close to the simulated 2.82. Parallelising only the solver is therefore not enough.
- **J3** also parallelises the copy and scales almost linearly up to 8 processors (97% efficiency on 4 and 92% on 8), reaching 11.96× on 16. It does not reach 16 for three reasons. First, the blocks have different sizes: border blocks skip the boundary rows and columns, so for

example `solve_3_3` executes 4 689 instructions against 7 543 for `solve_1_1`. Second, there is a barrier between the `solve` and `copy` phases. Third, the parent tasks spend some time creating their 16 children (the bundles of yellow lines at the start of each phase).

In the timelines, the yellow lines are the task creation and synchronisation edges drawn by the simulator: from the parent to its children at the start of each phase, and from the children to the join at the end. The green flags mark the start and end events of each task.

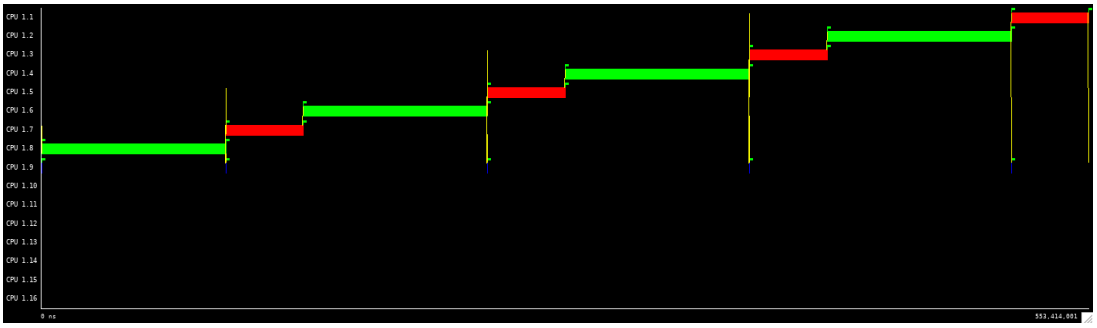
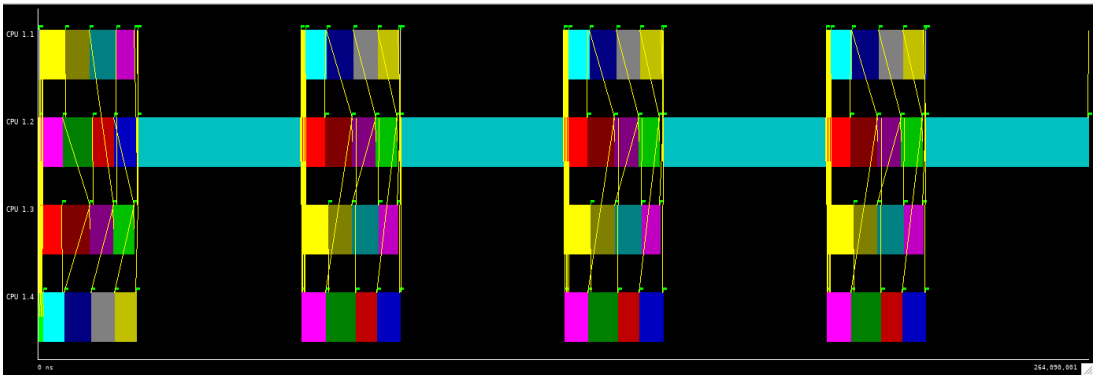
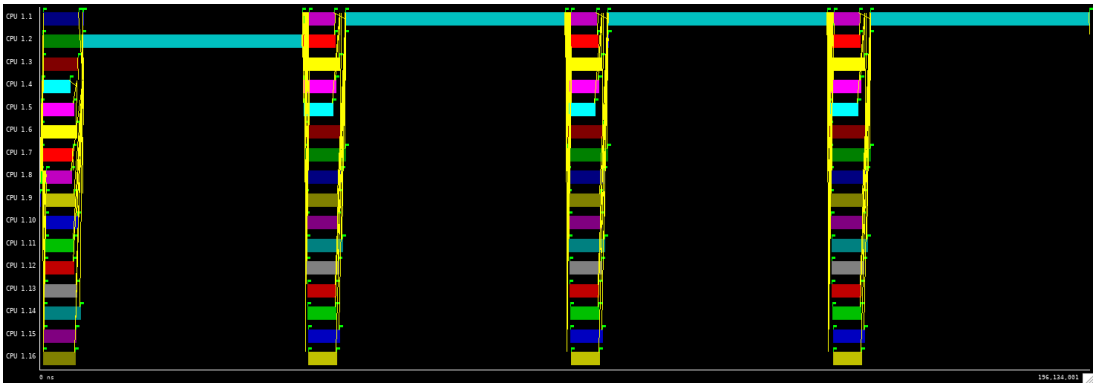


Figure 4: J1, 16 processors: `jacobi` (green) and `copy_mat` (red) never overlap.

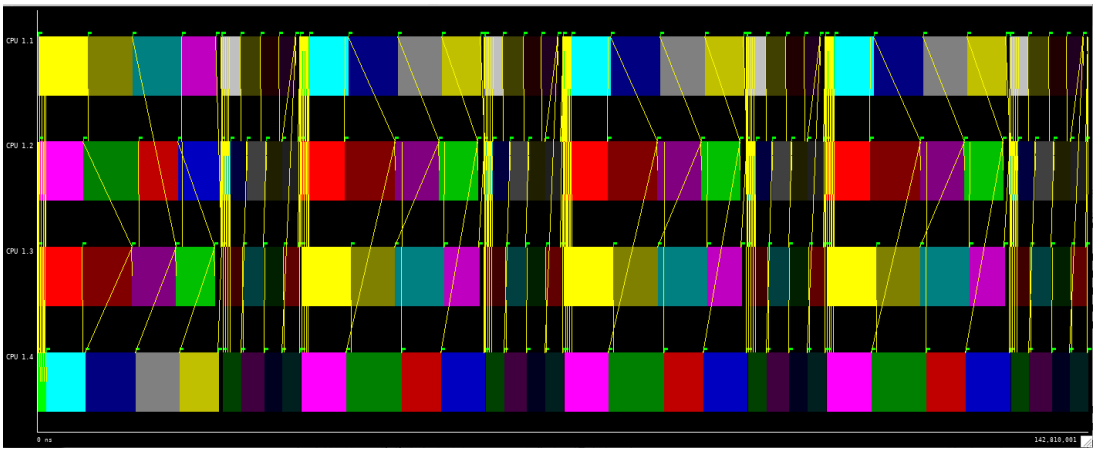


(a) 4 processors

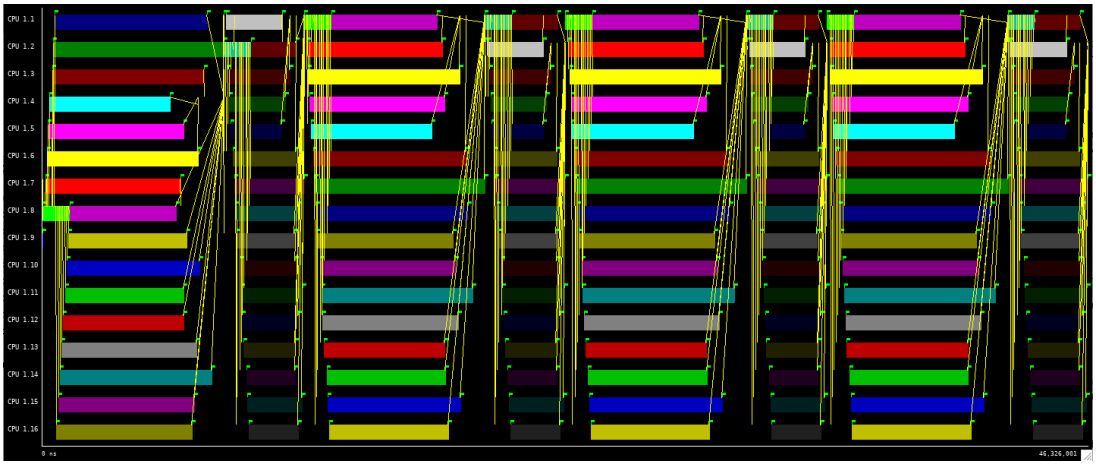


(b) 16 processors

Figure 5: J2: the 16 `solve` blocks run in parallel, but `copy_mat` (cyan) runs on a single processor while the others are idle.



(a) 4 processors



(b) 16 processors

Figure 6: J3: both phases are parallel. The bright blocks are `solve` and the dark blocks are `copy`.

Conclusions for the OpenMP implementation.

- The parallelism is inside each iteration, so the parallel region must be inside `solve` (and `copy_mat`), not around the `while` loop.
- The rows of `solve` are independent, so each thread can process its own rows without locks.
- `sum` needs a reduction, and the temporaries `tmp` and `diff` must be private.
- The phases must be separated by a barrier. The implicit barrier at the end of each parallel region provides it.
- `copy_mat` must also be parallelised. Otherwise it becomes the bottleneck, as in J2.

2.2 Gauss-Seidel

Gauss-Seidel updates `u` in place. To compute point (i, j) it needs the *new* values of the point above, $(i-1, j)$, and the point to the left, $(i, j-1)$, which have already been updated in the current iteration. It also needs the *old* values of the point below and the point to the right, which have not been updated yet. At block level, block (b_i, b_j) must therefore wait for the block above, $(b_i - 1, b_j)$, and the block to the left, $(b_i, b_j - 1)$. This is what makes the blocking provided in the code necessary: without blocks there would be a single task and no parallelism.

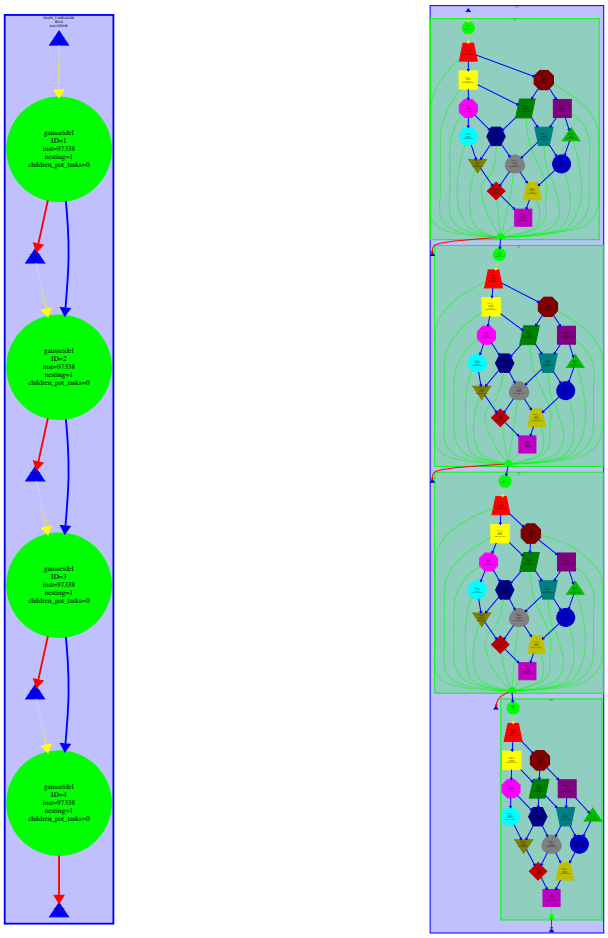
We explored three task decompositions. The source code of each version is stored in the directory `src/tareador-gauss/`.

G1 (coarse): one task per call to `solve` (`gausseidel` in `heat-tareador.c`).

G2 (blocks): G1 plus the 16 `solve(%d,%d)` block tasks, nested inside the `gausseidel` task. Each iteration must finish before the next one starts, as in the real code, where `solve` returns the residual that the `while` loop checks.

G3 (blocks, no barrier): only the 16 block tasks, without the outer `gausseidel` task. Consecutive iterations are allowed to overlap.

Task dependency graphs. G1 (Figure 7a) is again a sequential chain: each iteration reads and writes all of `u`. In G2 (Figure 7b), unlike Jacobi, *there are dependencies between the blocks of the same iteration*: every block has incoming edges from the block above and the block to the left. The blocks that can run at the same time are those on the same anti-diagonal ($b_i + b_j$ constant). With 4×4 blocks, the parallelism within one iteration grows $1 \to 2 \to 3 \to 4$ and then decreases $4 \to 3 \to 2 \to 1$ (a wavefront). In G3 (Figure 8) the iterations are no longer separated. Block $(0, 0)$ of iteration $t + 1$ only waits for blocks $(0, 1)$ and $(1, 0)$ of iteration t, which must read its old values before it overwrites them. The wavefronts of consecutive iterations therefore overlap, producing the diamond-shaped graph.



(a) G1 (coarse) (b) G2 (blocks)

Figure 7: TDGs of Gauss-Seidel, 4 iterations. In G2 each iteration contains a wavefront of dependent blocks.

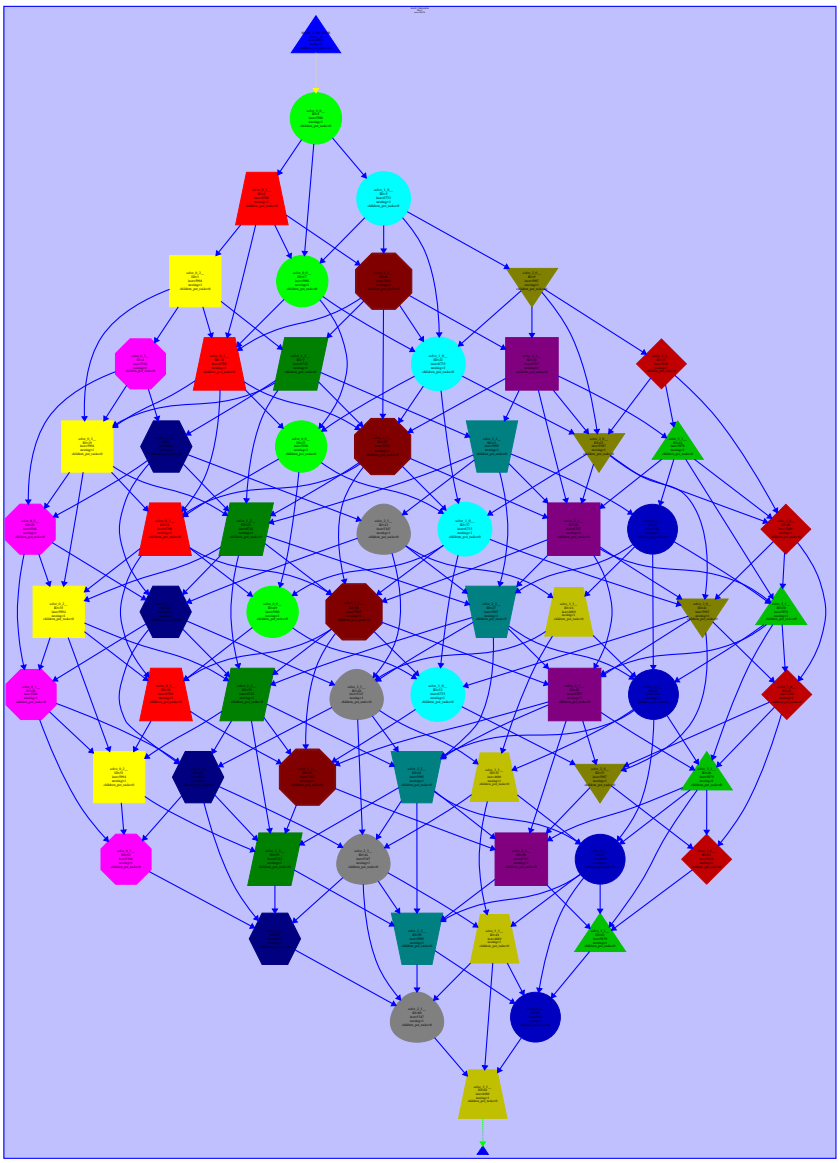


Figure 8: TDG of G3 (blocks, no barrier between iterations): the wavefronts of consecutive iterations overlap.

Simulated parallel execution. Table 2 shows the simulated time on 1, 4, 8 and 16 processors. Figures 9–11 show some of the timelines.

Version	1 proc.	4 proc.	8 proc.	16 proc.
G1 coarse	389.5 ms	389.5 ms	389.5 ms	389.5 ms
	1.00	1.00	1.00	1.00
G2 blocks	389.7 ms	172.9 ms	172.9 ms	172.9 ms
	1.00	2.25	2.25	2.25
G3 blocks, no barrier	389.7 ms	113.7 ms	86.1 ms	86.1 ms
	1.00	3.43	4.53	4.53

Table 2: Tareador simulation of Gauss-Seidel: simulated time and speedup with respect to 1 processor. The 1-processor times correspond to the total number of instructions of each graph.

- **G1** has no parallelism (Figure 9): one task at a time and a speedup of 1.

- **G2** reaches 2.25× and does not improve beyond 4 processors. The critical path of one iteration crosses 7 blocks, for example $(0,0) \to (0,1) \to (1,1) \to (1,2) \to (2,2) \to (2,3) \to (3,3)$, about 43 k of the 97 k instructions of the iteration. This bounds the speedup to about $97/43 \approx 2.3$. At most 4 blocks (one anti-diagonal) are ever ready at the same time, so processors 5 to 16 stay idle, and the 4, 8 and 16-processor executions take exactly the same time. Figure 10 shows the filling and draining of the wavefront in every iteration.

- **G3** reaches 4.53×. Each new wavefront starts about two block steps after the previous one, so the critical path of 4 iterations shrinks from $4 \times 7 = 28$ to about $7 + 3 \times 2 = 13$ block steps. It uses up to about 8 processors, so 16 processors give no further improvement. This overlap is *not* possible in the real code, because the `while` loop must check the residual of an iteration before starting the next. G2 is therefore the realistic model, and G3 shows the cost of that barrier.

The main conclusion is that the parallelism of Gauss-Seidel is limited by the number of blocks on an anti-diagonal and by the fill and drain of the wavefront in each iteration. Increasing the number of blocks in the j dimension (Section 5.3) shortens these phases relative to the total, and is the only way to use more threads.

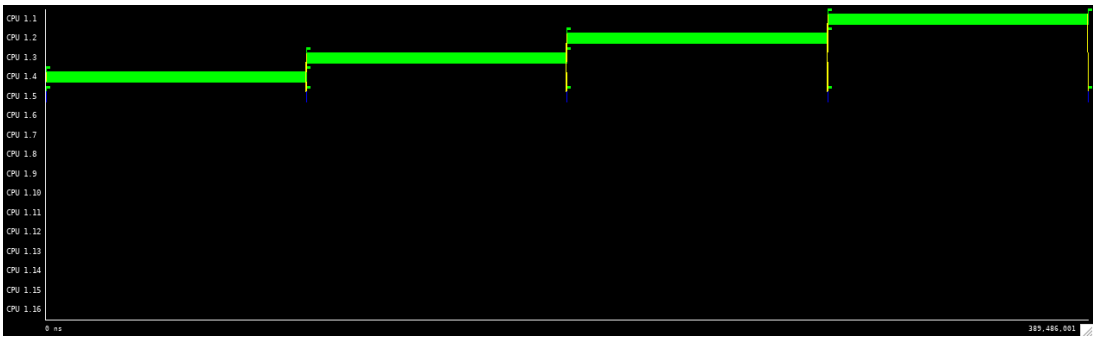
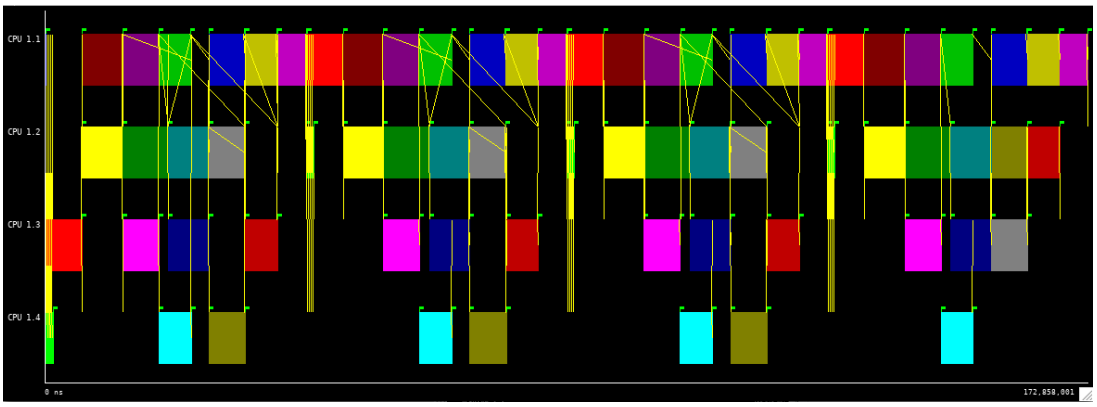
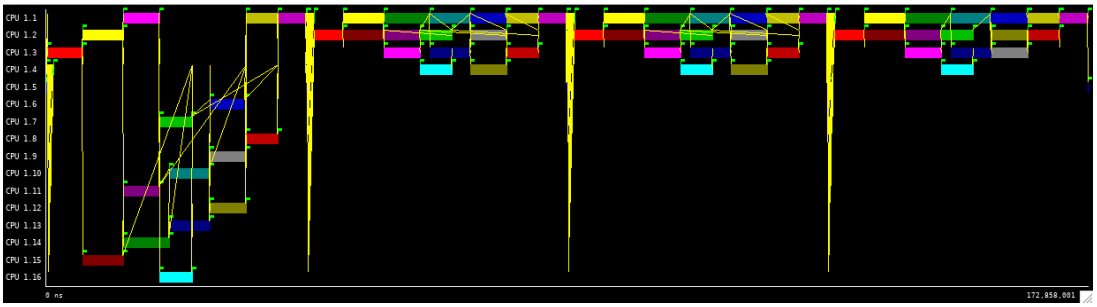


Figure 9: G1, 16 processors: one `gausseidel` task at a time.

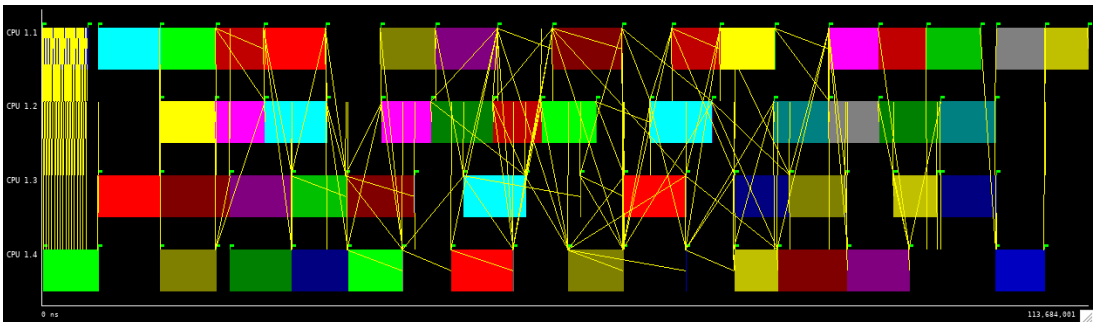


(a) 4 processors

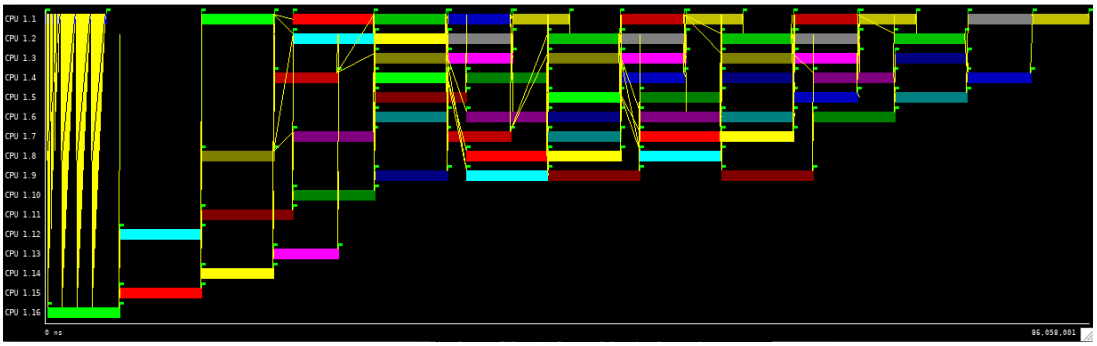


(b) 16 processors: same time as with 4, at most 4 processors busy after the first iteration

Figure 10: G2: wavefront execution; parallelism fills and drains in every iteration.



(a) 4 processors



(b) 16 processors

Figure 11: G3: without the barrier, consecutive iterations overlap and more processors are used.

3 Parallel Jacobi

3.1 Data decomposition implementation

The initial code in `solver-omp.c` is not correct:

```
#pragma omp parallel     // complete data sharing constructs here
for (int i = 1; i < sizex - 1; i++)
    for (int j = 1; j < sizey - 1; j++) { ... sum += diff * diff; unew
       [...] = tmp; }
```

`#pragma omp parallel` on its own does not distribute any work: every thread executes the complete loop. In addition, `tmp`, `diff` and `sum` are shared, so all threads overwrite the same temporaries (a data race), and the updates to `sum` are lost and counted multiple times. The residual is therefore wrong.

We completed the code following a geometric block decomposition by rows (Figure 4.1 of the assignment). Explicit work-sharing pragmas are not allowed, so each implicit task computes its own range of rows from `omp_get_thread_num()` and `omp_get_num_threads()`. We use the `lowerb`/`upperb` macros provided in the file, which also distribute the remainder rows when `sizex` is not a multiple of the number of threads. Clamping the range with `max(1, ...)` and `min(sizex-2, ...)` excludes the fixed boundary rows.

```
double solve (double *u, double *unew, unsigned sizex, unsigned sizey) {
    double tmp, diff, sum = 0.0;

    if (u == unew) return solve_gauss(u, sizex, sizey);

    #pragma omp parallel private(tmp, diff) reduction(+:sum)
    {
        int myid  = omp_get_thread_num();
        int nthr  = omp_get_num_threads();
        int i_start = lowerb(myid, nthr, sizex);
        int i_end   = upperb(myid, nthr, sizex);

        for (int i = max(1, i_start); i <= min(sizex - 2, i_end); i++) {
            for (int j = 1; j < sizey - 1; j++) {
                tmp = 0.25 * ( u[ i * sizey       + (j - 1) ] +
                               u[ i * sizey       + (j + 1) ] +
                               u[ (i - 1) * sizey + j       ] +
                               u[ (i + 1) * sizey + j       ] );
                diff = tmp - u[i * sizey + j];
                sum += diff * diff;
                unew[i * sizey + j] = tmp;
            }
        }
    }
    return sum;
}
```

The clauses follow from the dependency analysis:

- `private(tmp, diff)`: these are temporaries that are written before being read in every iteration, so each thread needs its own copy. Declaring them inside the loop would be equivalent.
- `reduction(+:sum)`: each thread accumulates into a private partial sum, and OpenMP combines the partial sums at the end of the region. This replaces the conflict on `sum` that we disabled in Tareador.

- No other synchronisation is needed. Each thread writes only its own rows of `unew` and only reads `u`. The implicit barrier at the end of the region guarantees that `copy_mat` starts once all rows have been computed, and that the reduction has finished before `sum` is returned.

We call this version **v1**; `copy_mat` is still sequential. We validated it with `submit-omp.sh` using 1 and 8 threads. The image is identical to the sequential one, and the number of iterations to convergence is the same.

3.1.1 Overall analysis

Table 3 shows the `modelfactors` results of v1 (`-s 1022 -n 1000`).

Overview of whole program execution metrics				
Number of processors	1	4	8	16
Elapsed time (sec)	2.40	1.57	1.45	1.54
Speedup	1.00	1.52	1.65	1.56
Efficiency	1.00	0.38	0.21	0.10

Overview of the Efficiency metrics in parallel fraction, ϕ=73.59%				
Number of processors	1	4	8	16
Global efficiency	99.83%	83.82%	62.34%	35.50%
Parallelization strategy efficiency	99.83%	90.54%	98.52%	97.90%
Load balancing	100.00%	91.93%	99.87%	99.88%
In execution efficiency	99.83%	98.50%	98.64%	98.02%
Scalability for computation tasks	100.00%	92.57%	63.28%	36.27%
IPC scalability	100.00%	92.96%	67.93%	42.75%
Instruction scalability	100.00%	99.98%	94.12%	85.89%
Frequency scalability	100.00%	99.60%	98.97%	98.76%

Statistics about implicit or explicit tasks in parallel fraction				
Number of processors	1	4	8	16
Number of implicit tasks per thread (average us)	1000.0	1000.0	1000.0	1000.0
Useful duration for implicit tasks (average us)	1762.37	475.96	348.13	303.73
Load balancing for implicit tasks	1.0	0.92	1.0	1.0
Time in synchronization implicit tasks (average us)	0.0	0.0	0.0	0.0
Time in fork/join implicit tasks (average us)	2.92	89.59	5.3	6.65

Table 3: `modelfactors` for v1 (parallel `solve`, sequential `copy_mat`).

Is the scalability appropriate? No. The speedup is only 1.52, 1.65 and 1.56 on 4, 8 and 16 threads (efficiency 0.10 on 16 threads), and the execution time hardly changes beyond 4 threads.

Which metric should be addressed first? The **parallel fraction**, which is only $\phi = 73.59\%$. More than a quarter of the sequential execution is outside any parallel region. By Amdahl's law, the speedup on 16 threads is limited to $1/(0.264 + 0.736/16) \approx 3.2$, whatever the efficiency of the parallel part.

The measured speedup (1.56) is even below that bound, because the parallel part itself scales poorly: the scalability for computation tasks is 36.3% on 16 threads, mainly due to the IPC scalability of 42.8%. The average useful duration of an implicit task drops from $1\,762\,\mu$s to $304\,\mu$s instead of the ideal $1762/16 \approx 110\,\mu$s. Because `copy_mat` runs on the master thread, every iteration the master writes all of `u` and reads all of `uhelp` in its own cache. In the next `solve`, the other threads (half of them on the other socket, since `KMP_AFFINITY=scatter`) must fetch their rows from the master's cache, and their writes to `uhelp` have to invalidate the master's copies. The data moves back and forth between cores every iteration, which lowers the IPC.

The unmodified template (v0). For comparison, we also traced the code exactly as provided (Table 4). It is slower than the sequential version for every number of threads (speedup 0.48 on 16 threads). Each implicit task lasts longer with 16 threads (3 293 μs) than the whole sequential `solve` (1 778 μs). This shows that every thread computes the complete matrix while all threads compete for memory bandwidth and for the cache line holding the shared `tmp`, `diff` and `sum`.

Overview of whole program execution metrics				
Number of processors	1	4	8	16
Elapsed time (sec)	2.43	3.92	3.56	5.06
Speedup	1.00	0.62	0.68	0.48
Efficiency	1.00	0.62	0.68	0.48

Overview of the Efficiency metrics in parallel fraction, ϕ=73.29%				
Number of processors	1	4	8	16
Global efficiency	99.85%	73.44%	78.46%	47.68%
Parallelization strategy efficiency	99.85%	92.01%	91.97%	88.31%
Load balancing	100.00%	98.01%	93.50%	92.44%
In execution efficiency	99.85%	93.89%	98.36%	95.53%
Scalability for computation tasks	100.00%	79.81%	85.31%	53.99%
IPC scalability	100.00%	79.86%	85.32%	53.97%
Instruction scalability	100.00%	100.00%	100.00%	100.00%
Frequency scalability	100.00%	99.94%	99.99%	100.04%

Statistics about implicit or explicit tasks in parallel fraction				
Number of processors	1	4	8	16
Number of implicit tasks per thread (average us)	1000.0	1000.0	1000.0	1000.0
Useful duration for implicit tasks (average us)	1777.86	2227.52	2084.0	3293.07
Load balancing for implicit tasks	1.0	0.98	0.94	0.92
Time in synchronization implicit tasks (average us)	0.0	0.0	0.0	0.0
Time in fork/join implicit tasks (average us)	2.65	232.97	103.53	200.64

Table 4: `modelfactors` for v0, the unmodified template (incorrect code, shown only for comparison).

3.1.2 Detailed analysis

Figure 12 shows the 16-thread trace of v1, zoomed in on about 26 iterations, with the *Instantaneous Parallelism* and *Implicit Tasks in Parallel Constructions* views synchronised.

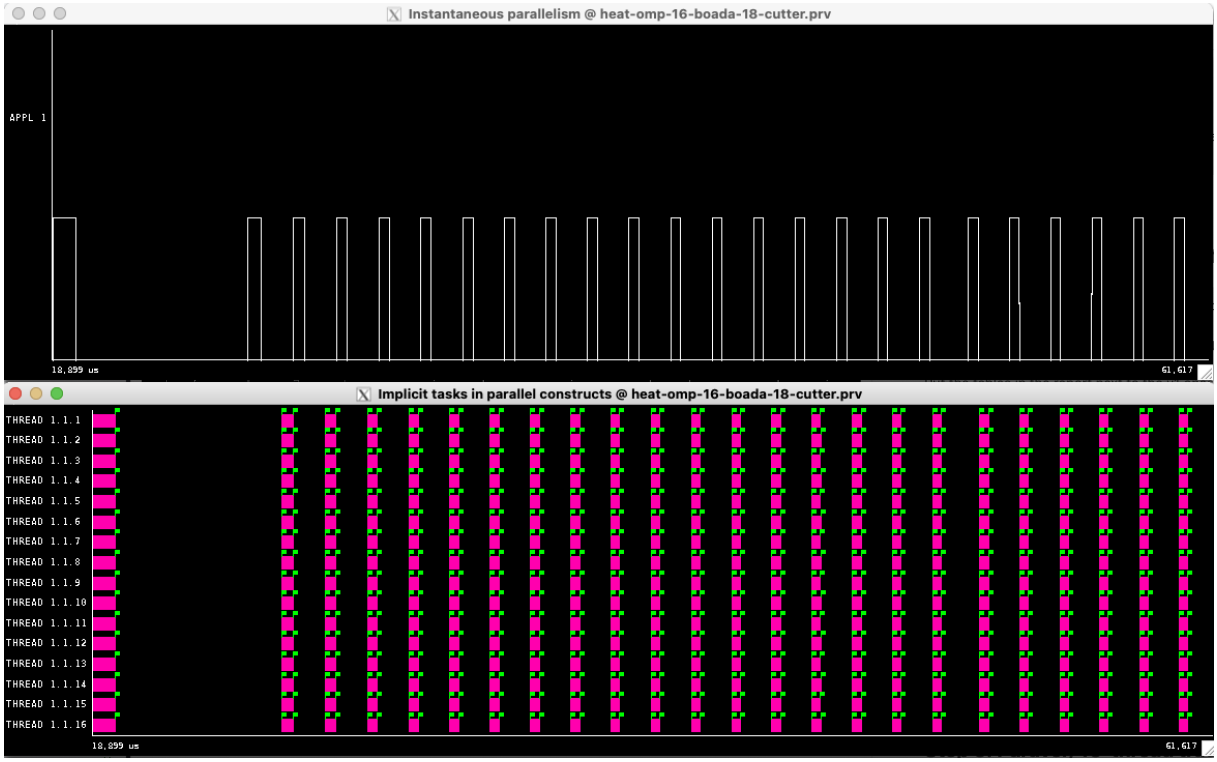


Figure 12: v1, 16 threads. Narrow pulses: parallel `solve` (all 16 threads). Wide gaps: sequential `copy_mat`, with instantaneous parallelism equal to 1.

Each iteration of about 1.5 ms has two parts:

- a short parallel region of about 0.5 ms, in which all 16 threads execute `solve` and the instantaneous parallelism is 16;
- a gap of about 1 ms, in which only the master thread runs and the instantaneous parallelism drops to 1.

The pulses are clean squares with no staircase at the end, so the load is well balanced (load balancing 99.9%). The longer gap after the first pulse is the warm-up of the first iteration.

Which region causes the low parallel fraction? It is what happens between two consecutive calls to the solver: `copy_mat(param.uhelp, param.u, np, np)` in `heat-omp.c`, which copies the new matrix back into `u` sequentially. With 16 threads, this serial gap is about twice as long as the parallel part. This is exactly what Tareador predicted for version J2, where `copy_mat` was the only task running between the parallel `solve` phases.

3.2 Optimisation

We parallelised `copy_mat` with the same row decomposition as `solve` (version **v2**):

```
void copy_mat (double *u, double *v, unsigned sizex, unsigned sizey) {
    #pragma omp parallel
    {
        int myid = omp_get_thread_num();
        int nthr = omp_get_num_threads();
        int i_start = lowerb(myid, nthr, sizex);
        int i_end   = upperb(myid, nthr, sizex);

        for (int i = max(1, i_start); i <= min(sizex - 2, i_end); i++)
            for (int j = 1; j < sizey - 1; j++)
```

```
                v[i * sizey + j] = u[i * sizey + j];
    }
}
```

The copied rows are independent (Tareador version J3), so no synchronisation is needed inside the region. Using the *same* decomposition matters: thread k copies exactly the rows it has just computed in `solve` (owner-computes rule), so the data is still in its own cache. With 16 threads, each thread's part of `u` and `uhelp` is about 1 MB, which fits in its 1.25 MB L2 cache. We validated v2 with 16 threads; the image is identical to the sequential one.

3.2.1 Overall analysis of the optimised code

Overview of whole program execution metrics				
Number of processors	1	4	8	16
Elapsed time (sec)	2.36	0.65	0.37	0.20
Speedup	1.00	3.63	6.30	11.78
Efficiency	1.00	0.91	0.79	0.74

Overview of the Efficiency metrics in parallel fraction, ϕ=98.74%				
Number of processors	1	4	8	16
Global efficiency	99.81%	93.75%	83.71%	83.28%
Parallelization strategy efficiency	99.81%	95.36%	95.67%	93.27%
Load balancing	100.00%	96.68%	98.68%	98.66%
In execution efficiency	99.81%	98.63%	96.95%	94.53%
Scalability for computation tasks	100.00%	98.32%	87.50%	89.29%
IPC scalability	100.00%	98.52%	90.25%	93.92%
Instruction scalability	100.00%	99.96%	97.73%	96.80%
Frequency scalability	100.00%	99.84%	99.20%	98.21%

Statistics about implicit or explicit tasks in parallel fraction				
Number of processors	1	4	8	16
Number of implicit tasks per thread (average us)	2000.0	2000.0	2000.0	2000.0
Useful duration for implicit tasks (average us)	1163.62	295.87	166.23	81.45
Load balancing for implicit tasks	1.0	0.97	0.99	0.99
Time in synchronization implicit tasks (average us)	0.0	0.0	0.0	0.0
Time in fork/join implicit tasks (average us)	2.26	21.43	8.6	7.58

Table 5: `modelfactors` for v2 (parallel `solve` and `copy_mat`).

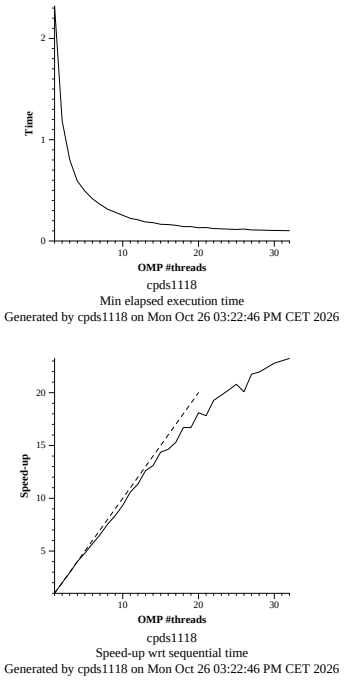


Figure 13: Strong scalability of v2 (`submit-strong-omp.sh`, Jacobi, 1 to 32 threads).

	v1	v2	v2 vs v1
Parallel fraction ϕ	73.59%	98.74%	
Elapsed time, 16 threads	1.54 s	0.20 s	7.7× faster
Speedup, 16 threads	1.56	11.78	
Efficiency, 16 threads	0.10	0.74	
IPC scalability, 16 threads	42.75%	93.92%	

Table 6: Comparison of the first implementation (v1) and the optimised one (v2).

- **Is the execution time reduced?** Yes. On 16 threads it drops from 1.54 s (v1) to 0.20 s (v2).
- **Has the scalability increased?** Yes. The speedup grows with the number of threads (3.63, 6.30 and 11.78 on 4, 8 and 16), and the strong-scalability plot (Figure 13) stays close to the ideal line up to about 12 threads, reaching about 23 on 32 threads. Beyond that point the gap to the ideal grows because the threads share the memory bandwidth (the stencil performs 5 loads and 1 store for every 4 additions). The plot measures a higher speedup than `modelfactors` (about 14.5 against 11.78 on 16 threads) because those executions are not instrumented and the script keeps the best of 3 runs. As the assignment notes, `time` is not very precise.
- **Is the parallel fraction larger?** Yes, it rises from 73.59% to 98.74%. The remaining 1.3% is the control of the `while` loop and the initialisation.
- **Speedup with respect to the first implementation on 16 threads:** $1.54/0.20 \approx 7.7\times$.

The other metrics are also good. Load balancing is 98.7% and the IPC scalability rises from 42.8% to 93.9%, because each thread now keeps its rows in its own cache. The time in fork/join

per thread is small (7.6 μs), even though there are now two parallel regions per iteration (2 000 implicit tasks per thread).

3.2.2 Detailed analysis of the optimised code

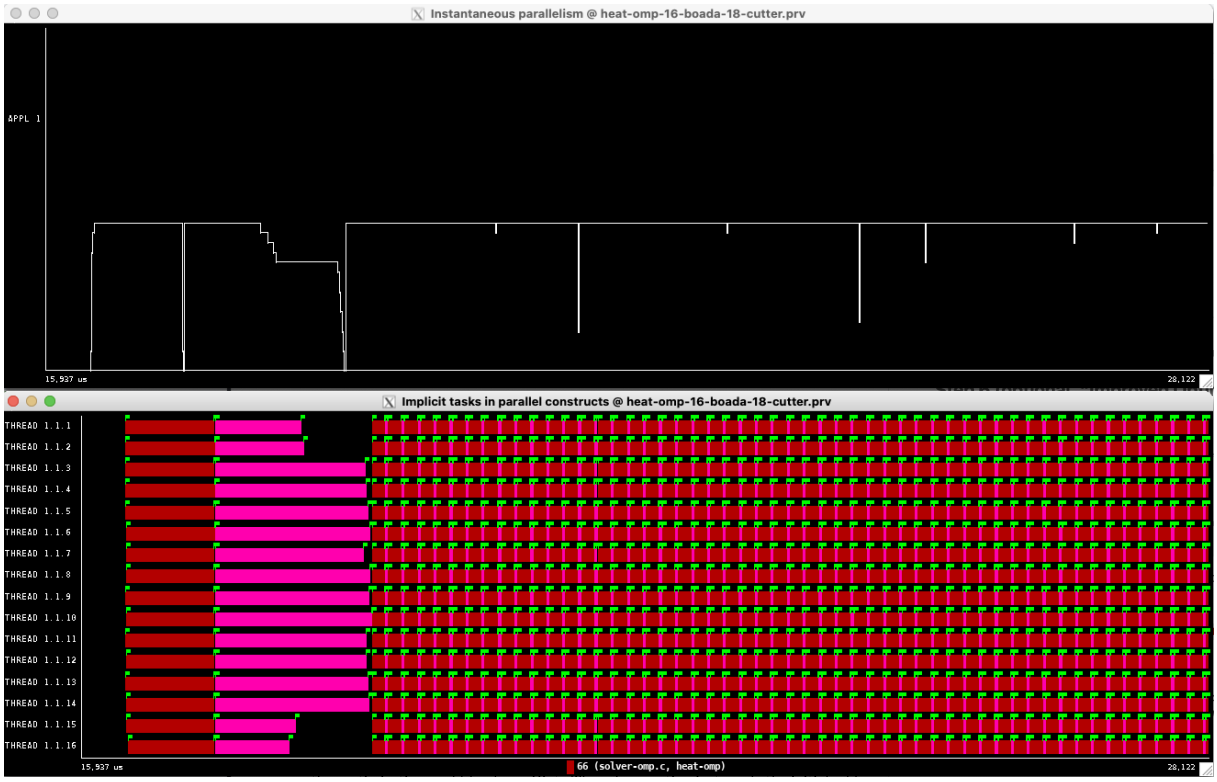


Figure 14: v2, 16 threads: `solve` (dark red) and the parallel `copy_mat` (magenta) alternate with only thin barriers between them. The left part is the first iteration (warm-up).

In Figure 14, after the first iteration, the two parallel regions of each iteration alternate on all 16 threads, and the instantaneous parallelism stays at 16 almost all the time, with only a few short drops at the barriers. About 50 iterations fit in 12 ms (about 0.23 ms per iteration, consistent with 0.20 s for 1 000 iterations), compared with about 1.5 ms per iteration in v1. The sequential gaps of Figure 12 have disappeared. The first iteration is slower and less balanced (some threads finish the first copy early) because the caches are still cold and memory pages are touched for the first time.

Further optimisation (not implemented). `copy_mat` only moves the new values back into `u`. Since `solve` only needs the addresses of the input and output matrices, the copy could be removed completely by swapping the `u` and `uhelp` pointers after each iteration. This replaces $O(N^2)$ work with $O(1)$ work and removes one parallel region and its barrier per iteration. It is safe because `uhelp` is initialised as a full copy of `u`, so both matrices hold the same boundary values. We have not implemented this version.